deque

A few points:

Deque is a data structure which stands for **double ended queue.** It is pronounced as **deck.**

This data structure has two ends and at each end, supports addition as well as deletion.

It is normally implemented as a doubly linked list. Otherwise at least one of the 4 operations would be inefficient requiring as many operations as the number of elements in the data structure.

In fact, the doubly linked list that we implemented with limited interface is a deque.

Once a data structure has been implemented, we do not directly modify it. We extend it by either inheritance in an object oriented language or by composition. As 'C' is not an object oriented language, we will limit our discussion to extending the use of data structure by composition.

We shall implement a stack using deque. We do not want to reinvent wheels. If already a data structure provides the required interface and implementation, we match out interface to that of the existing data structure.

Stack implementation using deque by composition :

This is the required stack interface.

```
void push(stack_t* ptr_stack, int key);

int pop(stack_t* ptr_stack);

int is_empty_stack(stack_t* ptr_stack);

int is_full_stack(stack_t* ptr_stack);



void init_stack(stack_t* ptr_stack);

void deinit_stack(stack_t* ptr_stack);
```

our stack data structure shall embed a deque implementation – in our example dll_t.

```
struct stack
{
        dll_t list_; // embedding or layering or containment or composition
};
typedef struct stack stack_t;
```

All our stack functions would invoke functions on the embedded object.

```
void push(stack_t* ptr_stack, int key)
{
	add_in_begin(&ptr_stack->list_, key);
}
int pop(stack_t* ptr_stack)
{
	return remove_in_begin(&ptr_stack->list_);
}
int is_empty_stack(stack_t* ptr_stack)
{
	return is_empty(&ptr_stack->list_);
}
int is_full_stack(stack_t* ptr_stack)
{
	return is_full(&ptr_stack->list_);
}
```

```
void init_stack(stack_t* ptr_stack)
{
	init(&ptr_stack->list_); // delegation
}
void deinit_stack(stack_t* ptr_stack)
{
	deinit(&ptr_stack->list_);
}
```

The functions of stack in turn delegate the work to our deque dll.c.

In C++ library, there are 4 primary data structures – array, vector, deque and list. The data structures like stack and queue are considered secondary data structures. These are implemented by embedding one of these primary data structures.